

DreamGraph Plugin Developer Manual

Guide and Reference Manual --- v12.1.0 Living DreamGraph

- Part I --- Plugin Developer Guide
- DreamGraph Plugin Developer Guide
 - Table of contents
- 1. Introduction
 - 1.1 What is a DreamGraph plugin?
 - 1.2 What a plugin is *not*
 - 1.3 Seams shipped through M6
 - 1.4 Trust posture in one paragraph
- 2. Quickstart — Hello Events in five minutes
 - 2.1 Prerequisites
 - 2.2 Allow in-process plugins (host switch)
 - 2.3 Copy the example into the instance
 - 2.4 Trust the plugin
 - 2.5 Inspect what loaded
 - 2.6 Call the contributed MCP tool
 - 2.7 Verify the closure-seam side-effects
 - 2.8 What you just exercised
- 3. Anatomy of a plugin
 - 3.1 Required files
 - 3.2 Module format — ESM only
 - 3.3 The entry contract
 - 3.4 Dependencies
 - 3.5 What the host passes you
- 4. Manifest and capabilities
 - 4.1 The intent-first manifest
 - 4.2 Why three orthogonal axes (capability, effect, intent)?
 - 4.3 The capability vocabulary
 - 4.4 The effect vocabulary
 - 4.5 Naming rules — both ids and tool names
 - 4.6 Engine version range
- 5. The PluginContext
 - 5.1 The allow surface
 - 5.2 Architect tabs and plan state
 - 5.3 The deny list
 - 5.4 The `signal`
 - 5.5 Logging
- 6. Seams: tools and resources
 - 6.1 Tools — `ctx.tools.register`
 - 6.2 Resources — `ctx.resources.register`
- 7. Seams: events
 - 7.1 Subscribe — `ctx.events.subscribe`
 - 7.2 Emit — `ctx.events.emit`
- 8. Closure seams: UI, policies, archetypes, markdown fences
 - 8.1 UI — `ctx.ui.register`
 - 8.2 Policies — `ctx.policies.propose`
 - 8.3 Archetypes — `ctx.archetypes.registerProvider`
 - 8.4 Markdown fences — `ctx.markdownFences.register`
- 9. Trust and security model
 - 9.1 The execution boundary
 - 9.2 The two safety switches
 - 9.3 The trust banner

- 9.4 The event-loop watchdog
 - 9.5 What "quarantine" means
 - 9.6 Coming in M8
- 10. Installation and lifecycle
 - 10.1 Discovery roots
 - 10.2 The five lifecycle states
 - 10.3 The CLI
 - 10.4 Hot reload
 - 10.5 Shipping inside a workspace
 - 10.6 Versioning and compatibility
- 11. Telemetry, debugging, and testing
 - 11.1 The plugin telemetry kinds
 - 11.2 Reading telemetry while developing
 - 11.3 Testing
 - 11.4 Common debugging recipes
- 12. Best practices and pitfalls
 - 12.1 Do
 - 12.2 Don't
 - 12.3 Pitfalls (compendium)
 - 12.4 The shipping checklist
- 13. Architect tabs
 - 13.1 Manifest declaration
 - 13.2 Registering a tab
 - 13.3 Browser routes
 - 13.4 Walkthrough
- Part II --- Plugin Reference Manual
- DreamGraph Plugin Reference Manual
 - Sections
 - Conventions
- Manifest schema (normative)
 - Top-level shape
 - Field-level constraints
 - Validation entry point
 - Example
- Capabilities (normative)
- Effects (normative)
 - Always-forbidden effects
 - Effect-to-seam matrix
- Reject reasons (normative)
 - Per seam
 - Host execution boundary
 - Helpers
- PluginContext API (normative)
 - Top-level
 - Surfaces
 - Seam definition shapes
 - Identity helpers
- Events (normative)
 - Envelope
 - Stable kinds
 - Experimental kinds
 - Selected payload schemas
 - Plugin-defined kinds
 - Replay
- dg plugin CLI (normative)
 - `dg plugin list <instance>`
 - `dg plugin inspect <instance> <plugin-id>`
 - `dg plugin trust <instance> <plugin-id>`
 - `dg plugin enable <instance> <plugin-id>`

- `dg plugin disable <instance> <plugin-id>`
- Notes
- Host configuration (normative)
 - Environment variables
 - `instance.json`
 - Discovery roots (recap)
- Telemetry events (normative)
 - Event kinds
 - Handler seams
 - Accepted output seams
 - Rejected output seams
 - Stability
- Architect tabs (normative)
 - Manifest declaration
 - Context API
 - Tab definition
 - Capabilities and effects
 - Lifecycle and telemetry
 - Browser contract

Part I --- Plugin Developer Guide

\newpage

DreamGraph Plugin Developer Guide

Audience: Engineers writing third-party plugins against `@dreamgraph/sdk`. **Engine baseline:** v12.1.0 "Living DreamGraph" (M0-M6 implemented; M5 webhooks shipped; M4 schedule actions deferred; Adaptive Future audit metadata available; standalone architect projections include pulse, playback, tension clusters, desires, and living-plan state). **Companion:** see the [Plugin Reference Manual](#) for strict, normative tables.

This guide is task-oriented. It walks you from "what is a plugin" through a working example, then explains every seam, the trust model, lifecycle, telemetry, testing, and the practices DreamGraph expects.

Table of contents

1. [Introduction](#) — what a plugin is and is not.
2. [Quickstart: Hello Events](#) — five-minute end-to-end install.
3. [Anatomy of a plugin](#) — files, packaging, ESM rules.
4. [Manifest and capabilities](#) — `plugin.json`, intent, effects.
5. [The PluginContext](#) — what `ctx` exposes (and what it deliberately does not).
6. [Seams: tools and resources](#) — the MCP-facing surface.
7. [Seams: events](#) — subscribe, emit, stable vs experimental.
8. [Seams: UI, policies, archetypes, markdown fences](#) — the M6 closure surface.
9. [Trust and security model](#) — in-process today, worker isolation tomorrow.
10. [Installation and lifecycle](#) — discovery, trust, restart, hot-disable.
11. [Telemetry, debugging, testing](#) — `plugin.*` events and the test harness.
12. [Best practices and pitfalls](#) — naming, naming, naming. And quotas.
13. [Architect tabs](#) — declarative standalone Architect tabs, explicit plan scope, persistence, and actions.

Building this guide locally as HTML/PDF: run `scripts/build-plugin-docs.ps1`. Outputs land under `docs/sdk/site/html/` and `docs/sdk/site/pdf/`.

1. Introduction

1.1 What is a DreamGraph plugin?

A **plugin** is a declarative + code package that extends a single DreamGraph instance through one or more **seams** — typed extension points exposed by the host. A plugin ships at minimum:

- a `plugin.json` manifest validated against the SDK schema, and
- an ESM entry module exporting a default `activate(ctx)` function.

When the host loads the plugin (in-process, see §9), it calls `activate(ctx)` once. Inside that function the plugin uses the `ctx.*` surfaces to register tools, resources, UI elements, policy proposals, archetype providers, markdown fences, and to subscribe to or emit events.

1.2 What a plugin is *not*

DreamGraph deliberately keeps the plugin contract narrow. A plugin **cannot**:

- reach the filesystem, network, or other instances directly through `ctx`;
- mutate the internal cognitive graph (`dream_graph.json`, `validated_edges.json`, `tension_log.json`, `event_log.json`);
- create or resolve tensions;
- subscribe to the host event bus from outside `ctx`;
- discover or interact with other plugins;
- invoke arbitrary `setInterval` / `setTimeout` schedulers (use schedule actions once M4 ships);
- run as a sandboxed process (M3 model is **trusted in-process**; see §9).

The full deny list lives in <docs/sdk/plugin-context.md> and in §5 of this guide.

1.3 Seams shipped through M6

Seam	Capability	Effect	Milestone
<code>tools</code>	<code>tools:register</code>	<code>emit_tool</code>	M3/M4
<code>resources</code>	<code>resources:register</code>	<code>emit_resource</code>	M3/M4
<code>events.subscribe</code>	<code>events:read</code>	<code>read_events</code> (implicit)	M3
<code>events.emit</code>	<code>events:emit</code>	<code>emit_event</code>	M3
<code>ui</code>	<code>ui:register</code>	<code>mutate_ui_registry</code>	M6
<code>policies</code>	<code>policy:propose</code>	<code>propose_policy</code>	M6
<code>archetypes</code>	<code>archetypes:provide</code>	<code>provide_archetypes</code>	M6
<code>markdownFences</code>	<code>markdown:register_fence</code>	<code>render_markdown_fence</code>	M6

Out of scope (post-M6):

- `webhooks:emit` — webhooks ship in M5 as a *core* subsystem, not a plugin seam. Plugins can register subscriptions only via the host CLI/MCP tools.
- `schedule:register_action` — the SDK type exists, but the host registry refactor (M4) is deferred. `defineScheduleAction` is reserved.
- `providers:register` — provider adapter seam is post-1.0.
- Executable iframe UI — the M6 builder is metadata-only.

1.4 Trust posture in one paragraph

Plugins run **in-process** inside the daemon. They share its heap and its Node permissions. The host can refuse to load a plugin (manifest invalid, capability not declared, host switch off, plugin not trusted), but it cannot prevent a malicious plugin from reaching `process` or `fs` through Node globals. **Treat installing a plugin like editing your `~/.bashrc`**. The full trust model is in §9.

\newpage

2. Quickstart — Hello Events in five minutes

This walkthrough installs the bundled `examples.hello-events` reference plugin into a running DreamGraph instance and exercises every seam. It is the fastest way to confirm your environment is correctly configured.

2.1 Prerequisites

- DreamGraph v9.0.0 or later, installed (`dg --version`).
- A live instance UUID (`dg list`).
- An MCP-capable client to call the contributed tool (the VS Code extension, Claude Desktop, `mcp_inspector`, or a script using `@modelcontextprotocol/sdk`).

2.2 Allow in-process plugins (host switch)

In-process plugin execution is **deny-by-default**. Add the line to your instance's `engine.env` (or set the env var before launching the daemon):

```
DG_ALLOW_INPROCESS_PLUGINS=true
```

The switch and its caveats are documented in all three `engine.env` templates ([default](#), [ollama](#), [lmstudio](#)).

2.3 Copy the example into the instance

```
$uuid = "<your-instance-uuid>"
Copy-Item -Recurse examples/hello-events `
"$env:USERPROFILE\.dreamgraph\instances\$uuid\plugins\examples.hello-events"
```

DreamGraph discovers plugins from two roots only:

1. `<instance>/plugins/*/plugin.json`
2. `instance.json:plugins[]` (declared paths)

There is no auto-discovery from arbitrary `node_modules`.

2.4 Trust the plugin

```
dg plugin trust dreamgraph examples.hello-events
dg restart dreamgraph
```

`dg plugin trust` flips the `trusted: true` flag in `instance.json:plugins[]`. A plugin must be trusted **and** the host switch must be on for in-process loading to occur.

On startup you will see a one-line trust banner per loaded plugin:

```
plugin examples.hello-events@0.2.0 loaded with FULL HOST TRUST (in-process)
```

2.5 Inspect what loaded

```
dg plugin list dreamgraph
dg plugin inspect dreamgraph examples.hello-events
```

The host exposes a live `system://plugins` MCP resource with the same data: status, intent, runtime, declared and last-seen effects.

2.6 Call the contributed MCP tool

DreamGraph does not ship a CLI shortcut for invoking tools. Use any MCP client. The bundled probe script does it via the streamable HTTP transport:

```
node scripts/probe-hello-events.mjs
```

Expected output:

```
tool examples_hello_events_greet → { "greeting": "Hello, world!" }
resource plugin://examples.hello-events/manifest → { id: "...", version: "0.2.0", seams: [...] }
```

Note: exposed tool names must use model-safe characters. The example tool is `examples_hello_events_greet`, not the older dotted form.

2.7 Verify the closure-seam side-effects

Each closure seam writes through a host-locked store and prunes on unload:

```
$data = "$env:USERPROFILE\.dreamgraph\instances\$uuid\data"
Get-Content "$data\ui_registry.json"          # contains examples.hello-events.greeting
Get-Content "$data\plugin_policy_proposals.json" # contains source: plugin:examples.hello-events
```

Disable the plugin and watch every contribution disappear:

```
dg plugin disable dreamgraph examples.hello-events
```

The UI element, policy proposal, archetype provider, and markdown fence are all removed automatically.

2.8 What you just exercised

Seam	Where to look
<code>events.subscribe</code>	<code>dg plugin inspect</code> shows declared <code>events:read</code> + a subscriber on <code>snapshot.changed</code> .
<code>events.emit</code>	The <code>examples.hello-events.activated</code> event appears on the bus / SSE stream.
<code>tools.register</code>	<code>tools/list</code> over MCP shows <code>examples_hello_events_greet</code> .
<code>resources.register</code>	<code>resources/list</code> shows <code>plugin://examples.hello-events/manifest</code> .

Seam	Where to look
<code>ui.register</code>	<code>ui_registry.json</code> gains a tagged entry.
<code>policies.propose</code>	<code>plugin_policy_proposals.json</code> gains a <code>source: plugin:examples.hello-events</code> entry.
<code>archetypes.registerProvider</code>	<code>system://plugins</code> lists the inline provider.
<code>markdownFences.register</code>	The webview SDK can introspect language <code>dg-hello</code> .

\newpage

3. Anatomy of a plugin

3.1 Required files

```
my-plugin/
├─ plugin.json      # manifest, validated by @dreamgraph/sdk
├─ index.js         # ESM entry (path declared as `main` in manifest)
├─ package.json     # optional but recommended; "type": "module"
└─ README.md        # optional; appears next to dg plugin inspect
```

Anything else (build artifacts, fixtures, sub-modules) is fine. The host loads **only** the file referenced by `manifest.main`. Imports inside that file are resolved by Node's standard ESM resolver against the plugin folder.

3.2 Module format — ESM only

The host loads plugins via dynamic `import()`. Every plugin entry MUST be an ES module. CommonJS is not supported. Either:

- Set `"type": "module"` in the plugin's `package.json`, or
- Use the `.mjs` extension for the entry file.

If you write the plugin in TypeScript, ship the compiled JavaScript and point `main` at the build output:

```
{ "main": "./dist/index.js" }
```

3.3 The entry contract

The entry module must default-export `activate(ctx)`:

```
export const meta = { id: "acme.changelog", version: "0.1.0" };

export default function activate(ctx) {
  ctx.logger.info("activating");

  // Register seams synchronously inside activate(). Returning a promise is
  // allowed but the host counts the plugin as "loaded" once activate() resolves.
  ctx.tools.register({ /* ... */ });

  return {
    deactivate() {
      // Optional. The host calls this on dg plugin disable / instance shutdown.
    }
  };
}
```

```

    // Use it to flush in-flight state. Contributions registered through ctx
    // are pruned automatically – you do not need to call unregister manually.
  },
};
}

```

`activate(ctx)` may also be `async`. The host awaits it. If `activate` throws (or the returned promise rejects), the host emits `plugin.errorred` with `reason: "handler_threw"` and the plugin is quarantined.

3.4 Dependencies

The SDK ships with **zero runtime dependencies** beyond `zod` (which the daemon already bundles). For the M3 trusted-in-process model:

- You **may** use any npm package. The host does not constrain `node_modules`.
- You **should** keep dependencies small and well-known. Plugin dependencies live in the daemon's process and inherit its trust.
- You **must not** depend on the daemon's internals (`src/...`). The only supported import is `@dreamgraph/sdk`. The daemon's source is not re-exported and may change between releases.

3.5 What the host passes you

`activate(ctx)` receives a fully constructed `PluginContext`. The full shape is documented in [§5 The PluginContext](#). The shortlist:

- `ctx.plugin.{id,version}`
- `ctx.instance.uuid`
- `ctx.logger.{debug,info,warn,error}`
- `ctx.events.{subscribe,emit}`
- `ctx.tools.register`
- `ctx.resources.register`
- `ctx.ui.register`
- `ctx.policies.propose`
- `ctx.archetypes.registerProvider`
- `ctx.markdownFences.register`
- `ctx.signal` — `AbortSignal`, aborted on disable / handler timeout

\newpage

4. Manifest and capabilities

4.1 The intent-first manifest

Every plugin ships a `plugin.json` validated against `PluginManifestSchema` ([source](#)). The schema is **intent-first**: it asks not only *what permissions you want* (capabilities) but *what observable effects you claim you will have* (`expectedEffects`) and *what you guarantee not to do* (`forbiddenEffects`).

```

{
  "id": "acme.changelog",
  "version": "0.1.0",
  "displayName": "Acme Changelog",
  "description": "Weekly changelogs from validated edges.",
  "engine": { "dreamgraph": ">=9.0.0" },
  "main": "./dist/index.js",

```

```

"intent": "Summarize validated cognitive changes into weekly changelogs.",
"expectedEffects": ["read_events", "emit_resource", "emit_tool"],
"forbiddenEffects": ["write_internal_graph", "modify_tensions"],

"capabilities": ["events:read", "tools:register", "resources:register"],

"tools":      [{ "name": "acme.changelog.summarize_week" }],
"resources":  [{ "uriNamespace": "plugin://acme.changelog/latest" }],
"ui":         [],
"policies":   [],
"archetypeProviders": [],
"markdownFences": [],

"policy": { "phasePermissions": ["analysis"], "writableFiles": [] }
}

```

4.2 Why three orthogonal axes (capability, effect, intent)?

Axis	Question it answers	Enforced where
capabilities	<i>Are you allowed to use the API?</i>	At seam call time (gate function)
expectedEffects	<i>What do you say you will do?</i>	At every host-mediated effect
forbiddenEffects	<i>What do you guarantee NOT to do?</i>	Hard stop, even if capability allows
intent	<i>Why are you doing it?</i>	Operator review + telemetry

A plugin that performs an effect outside `expectedEffects` emits `plugin.output.rejected{ reason: "effect_undeclared" }` and is auto-quarantined. A plugin that hits a `forbiddenEffects` entry is refused even if the matching capability would otherwise allow it.

4.3 The capability vocabulary

Locked for v1 — see [reference / capabilities](#):

Capability	Allows
events:read	subscribe to stable event kinds
events:read:experimental	subscribe to experimental kinds (instance opt-in)
events:emit	publish via <code>ctx.events.emit()</code>
tools:register	<code>defineTool</code> registration
resources:register	<code>defineResource</code> registration
resources:read	(reserved) read other plugins' resources
ui:register	metadata-only UI element registration
policy:propose	append discipline rules tagged <code>plugin:<id></code>
schedule:register_action	(reserved, M4) custom schedule actions
webhooks:emit	(reserved, post-MVP) plugin-driven outbound HTTP
archetypes:provide	register a federation archetype provider
providers:register	(reserved, post-1.0) provider adapter seam

Capability	Allows
<code>markdown:register_fence</code>	register a webview markdown fence language
<code>architect:register_tab</code>	register a manifest-declared standalone Architect tab
<code>architect:read_plan</code>	read explicit plan-bound Architect state
<code>architect:write_plan_state</code>	persist daemon-owned namespaced Architect state
<code>architect:register_sidebar_summary</code>	project Architect sidebar summary and badges

A capability not present in the manifest's `capabilities` array means the matching seam call returns `*_capability_missing` and is dropped.

4.4 The effect vocabulary

Effect	Meaning
<code>read_events</code>	subscribe to the bus
<code>read_internal_graph</code>	read internal-tier files (graph, validated edges, tensions)
<code>read_external_state</code>	read external-tier files (features, archetypes, ADRs, UI registry)
<code>write_internal_graph</code>	mutate the graph
<code>modify_tensions</code>	create/resolve tensions
<code>emit_resource</code>	register/update an MCP resource
<code>emit_tool</code>	register an MCP tool
<code>emit_event</code>	publish events
<code>emit_webhook</code>	(reserved) outbound HTTP via webhook worker
<code>register_schedule_action</code>	(reserved) custom schedule action
<code>invoke_llm</code>	call the host LLM proxy
<code>mutate_ui_registry</code>	register/update UI elements
<code>propose_policy</code>	append discipline rules
<code>provide_archetypes</code>	feed federation archetypes
<code>render_markdown_fence</code>	declare a webview markdown fence
<code>register_provider_adapter</code>	(reserved) provider adapter
<code>render_architect_tab</code>	project a host-rendered standalone Architect tab
<code>read_architect_plan_projection</code>	load an explicit plan-bound Architect snapshot
<code>write_architect_plan_state</code>	persist daemon-owned namespaced Architect state
<code>render_architect_sidebar_summary</code>	project sidebar summary and badge data

4.5 Naming rules — both ids and tool names

Surface	Pattern	Reject reason
<code>id</code>	<code>^[a-z0-9][a-z0-9.-]*[a-z0-9]\$</code> (DNS-like)	<code>manifest_invalid</code>

Surface	Pattern	Reject reason
Tool name	<code><plugin-id>.<tool_name></code> (lowercase, dot-prefixed)	<code>tool_name_unprefixed</code>
Resource uri	<code>plugin://<plugin-id>/...</code>	<code>resource_uri_out_of_namespace</code>
UI element id	starts with <code><plugin-id>.</code>	<code>ui_id_unprefixed</code>
Policy proposal id	lowercase <code>[a-z0-9._-]+</code>	<code>manifest_invalid</code>
Markdown fence lang	lowercase <code>[a-z0-9._-]+</code> , no collision	<code>markdown_fence_language_invalid</code> / <code>_collision</code>
Architect tab id	starts with <code><plugin-id>.</code> ; runtime declaration matches manifest	<code>architect_tab_id_unprefixed</code> / <code>architect_tab_undeclared</code>

Per ADR-128, tool names must carry primary semantics: a model should be able to infer the dominant action and target from the name alone, without reading the description. Prefer `summarize_week` over `process_data`.

4.6 Engine version range

`engine.dreamgraph` is a semver range. The host refuses to load plugins whose range excludes the current engine version (`engine_version_mismatch`). Ship the narrowest range that actually works — `>=9.0.0` is appropriate for v1 plugins.

\newpage

5. The PluginContext

`ctx` is the single object the host hands you. Its shape is fixed — see [reference / context-api](#) for the strict TypeScript signatures.

5.1 The allow surface

```
interface PluginContext {
  readonly plugin: { id: string; version: string };
  readonly instance: { uuid: string };
  readonly logger: PluginLogger; // namespaced, mirrored to event bus
  readonly events: { subscribe; emit };
  readonly tools: { register };
  readonly resources: { register };
  readonly ui: { register };
  readonly policies: { propose };
  readonly archetypes: { registerProvider };
  readonly markdownFences: { register };
  readonly architect: { tabs: { register }; planState: { read; write } };
  readonly signal: AbortSignal; // aborted on disable / timeout
}
```

Every register / propose / subscribe call returns an `unregister` / `unsubscribe` function. **You do not need to call it on shutdown** — the host prunes all contributions automatically when the plugin is disabled or the instance shuts down. Use the returned functions only when you want a contribution to disappear *before* deactivation (rare).

5.2 Architect tabs and plan state

`ctx.architect.tabs.register()` contributes a manifest-declared standalone Architect tab rendered by the host. `ctx.architect.planState.read()` and `.write()` access daemon-owned namespaced state with explicit plan ids and revision tokens. Browser clients receive validated descriptors and snapshots only. See [Architect tabs](#).

`ctx.ui.register()` remains a separate metadata-only semantic UI registry seam; it does not execute browser code.

5.3 The deny list

`ctx` deliberately does **not** expose:

- Raw `fs`, `process`, `require`, `dynamic import()`.
- Any filesystem path (no `dataDir`, no instance root).
- Internal-tier writers (`dream_graph.json`, `validated_edges.json`, `tension_log.json`, `event_log.json`).
- Tension mutation (`ctx.tensions.create`, `.resolve`).
- Graph mutation (`ctx.graph.promoteCandidate`, `.editEdge`).
- Other plugins' loggers, configs, or quotas.
- `setInterval` / `setTimeout` helpers — long-running or recurring work must use schedule actions (post-M4).
- Network primitives — outbound HTTP goes through the (operator-owned) webhook worker, not through `ctx`.

This is an **API boundary**, not a runtime sandbox. M3 plugins run in the daemon's process and *can* reach Node globals if they want to. Doing so is a trust violation and grounds for quarantine, not a sandbox escape. See §9 for the full trust narrative.

5.4 The `signal`

`ctx.signal` is an `AbortSignal` the host aborts when:

- the plugin is disabled (`dg plugin disable`),
- the instance shuts down,
- a handler exceeds the timeout budget,
- the plugin is quarantined for any other reason.

Pass it to long-running work that supports cancellation (`fetch(url, { signal })`, your own loops). You are not required to use it, but ignoring it means your plugin can keep running for a few extra seconds during shutdown.

5.5 Logging

`ctx.logger` is namespaced under `plugin:<id>`. Lines appear in the daemon's log stream and are mirrored to the event bus as standard log events. Do **not** use `console.log` / `console.error` — those bypass the bus and the operator may not see them.

\newpage

6. Seams: tools and resources

These two seams ship plugin output over MCP. They are the most common reason to write a plugin.

6.1 Tools — `ctx.tools.register`

```
ctx.tools.register({
  name: "acme.changelog.summarize_week", // MUST start with "<plugin-id>."
  description: "Summarize the last N days of validated edges.",
  inputSchema: {
    type: "object",
    properties: { days: { type: "integer", minimum: 1 } },
  },
});
```

```

    required: ["days"],
  },
  expectedEffects: ["emit_tool"],
  handler: async ({ days }, _extra) => {
    return { summary: `Last ${days} days...` };
  },
});

```

Naming (mandatory)

The tool `name` must:

- be lowercase,
- start with the plugin id followed by `.`,
- carry the primary semantics of the operation (ADR-128).

`acme.changelog.summarize_week` ✓ `acme.changelog.process` ✗ (no semantic verb-target) `summarize_week` ✗
(no plugin prefix → `tool_name_unprefixed`)

Input schema

You write a JSON Schema, the host converts it to a Zod shape and uses that to validate `args` before calling your handler.
Supported keywords today:

- `type`: "object" | "string" | "number" | "integer" | "boolean" | "array"
- `properties`, `required`, `items`, `enum` (string only),
- `description` (mapped to `.describe()`).

Anything else falls back to `z.unknown()`.

What the host does for you

- Registers the tool with the MCP `Server.tool()` factory.
- Validates incoming arguments against your schema; invalid args never reach your handler.
- Wraps your handler in `plugin.handler.started` / `plugin.handler.completed` telemetry with a correlation id.
- Emits `plugin.output.accepted` if the host successfully forwarded your output, or `plugin.output.rejected{ reason }` if your handler threw, the effect was undeclared, or the tool name was not prefixed.

What you owe the host

- Return JSON-serializable values from `handler`. The MCP transport encodes the return value as the tool's `content`.
- Honor `ctx.signal` for any long-running work.
- Keep your handler under the per-plugin handler timeout (default 5 s for in-process; configurable). A handler that hangs is killed and the plugin is quarantined with `handler_timeout`.

6.2 Resources — `ctx.resources.register`

```

ctx.resources.register({
  uriNamespace: "plugin://acme.changelog/latest",
  name: "Latest weekly changelog",
  description: "Returns the most recent rendered summary.",
  expectedEffects: ["emit_resource"],
  handler: async () => {
    rendered_at: new Date().toISOString(),
    body: "## Week ending ...",
  },
});

```

URI rules

- MUST start with `plugin://<plugin-id>/`. Anything else → `resource_out_of_namespace`.
- The namespace is exact: `plugin://acme.changelog/latest` and `plugin://acme.changelog/by-id/123` are two distinct registrations.
- Two plugins cannot collide; the prefix is enforced.

What the host does

- Exposes the resource over MCP `resources/list` and `resources/read`.
- Routes reads to your handler; the return value is JSON-serialized.
- Emits the same `plugin.output.{accepted,rejected}` telemetry as tools.

\newpage

7. Seams: events

Events are the cognitive heartbeat. The SDK exposes two operations: subscribe to host-emitted events, and emit your own.

7.1 Subscribe — `ctx.events.subscribe`

```
const off = ctx.events.subscribe("snapshot.changed", (evt) => {
  ctx.logger.info(`observed ${evt.kind}`, evt.payload);
});
```

Stable vs experimental

Set	Capability	Import from
Stable kinds	<code>events:read</code>	<code>@dreamgraph/sdk</code>
Experimental	<code>events:read:experimental</code>	<code>@dreamgraph/sdk/events/experimental</code>

The split is **type-enforced** in the SDK: the default `defineEventHandler` factory is generic over `StableEventKind`. To subscribe to an experimental kind you must use `defineExperimentalEventHandler` and the host must have `experimental_plugins: true` in `instance.json`. Subscribing to an experimental kind without the capability returns `event_kind_experimental`; subscribing to a kind not declared in the manifest returns `event_kind_not_declared`.

Bus model

The DreamGraph event bus is **synchronous**: the producer call site does not await your handler. The host wraps your handler in a deferred queue so a slow subscriber cannot stall cognition. Therefore:

- Your handler MAY be async; the queue runner awaits it.
- Your handler MUST tolerate handler-level timeouts (default 5 s). A timeout emits `handler_timeout` and quarantines the plugin.
- Your handler runs in the daemon's heap. Long synchronous CPU work will be detected by the event-loop watchdog (`event_loop_stall`, see §9).

7.2 Emit — `ctx.events.emit`

```
ctx.events.emit("examples.hello-events.activated", {
  at: new Date().toISOString(),
});
```

Rules

- The kind name MUST NOT collide with a built-in `GraphEventKind`. Conventional practice is `<plugin-id>.<event_name>` (matches the tool/resource pattern).
- Emitting a kind requires the `events:emit` capability **and** `emit_event` in `expectedEffects`. Missing either → `effect_undeclared` or `emit_capability_missing`.
- The payload is JSON-serialized and broadcast on the bus + SSE stream (`/explorer/api/events`) + matching webhook subscriptions.

Replay window

The SSE stream maintains a 500-event ring buffer; the file-backed `event_log.json` is the long-term record. Plugins should treat the bus as **at-most-once**: if your subscriber missed an event because your plugin was disabled or the daemon restarted, the SSE replay window does not extend back indefinitely. Critical state must be reconstructed from a resource read or by reading the event log.

\newpage

8. Closure seams: UI, policies, archetypes, markdown fences

These four seams are the M6 closure. Each is **metadata-only** and writes through a host-locked store with automatic prune-on-unload.

8.1 UI — `ctx.ui.register`

```
ctx.ui.register({
  id: "acme.changelog.summary_panel",      // MUST start with "<plugin-id>."
  name: "Weekly Summary Panel",
  purpose: "Display the latest weekly changelog body.",
  category: "data_display",
  inputs: [
    { name: "week_id", type: "string", required: true },
  ],
  outputs: [],
  interactions: [],
  implementations: [
    { platform: "react", component: "WeeklySummaryPanel" },
  ],
});
```

The host validates the element against the live `ui_registry.json` schema and writes it with provenance `plugin:<plugin-id>`. The metadata is consumable by the explorer and by external dashboards reading `ui_registry.json`.

Out of scope for v1: an executable iframe panel runtime. M6 ships the metadata seam only; the iframe contract is post-1.0.

8.2 Policies — `ctx.policies.propose`

```
ctx.policies.propose({
  id: "weekly-changelog-allowed-during-execution",
  title: "Weekly changelog generation is allowed during the execution phase",
  rationale: "The summary tool is a read-only synthesis with no side-effects.",
  applies_to: ["plugin"],
  phases: [{ phase: "execution", permission: "allowed" }],
});
```

```
    severity: "info",
  });
```

Plugins **propose** discipline rules; they cannot **weaken** existing rules. A proposal that contradicts a stricter built-in rule is rejected with `policy_weakening_rejected`. Accepted proposals land in `plugin_policy_proposals.json` tagged `source: "plugin:<id>"`. The merge into the live `discipline://manifest` happens on a separate, operator-reviewed path.

8.3 Archetypes — `ctx.archetypes.registerProvider`

```
ctx.archetypes.registerProvider({
  id: "starter-pack",
  name: "Starter Archetype Pack",
  inline: {
    source: "acme.changelog",
    version: "1",
    archetypes: [
      { id: "the-weekly-recap", name: "The Weekly Recap", summary: "...", tags: ["example"] },
    ],
  },
});
```

Two provider modes:

Mode	Field	When polled
<code>inline</code>	<code>inline</code>	Immediately at registration; static feed.
<code>remote</code>	<code>url</code> , <code>etag</code>	(Reserved) polled by the federation poller.

For v1, prefer `inline`. Live polling is exercised separately by federation tests and is not yet part of the plugin contract.

8.4 Markdown fences — `ctx.markdownFences.register`

```
ctx.markdownFences.register({
  language: "acme-changelog",
  label: "Acme Changelog",
  description: "Renders a structured changelog block.",
  platforms: ["webview"],
});
```

This is a **manifest-only stub** in the host: it tells the webview SDK that a fence language exists and describes its rendering surface, but the actual renderer lives in a webview-side package (`window.registerCardFencePlugin`). The closure here gives operators visibility — they can see which plugins claim which fence languages — and reserves the language so two plugins cannot collide (`markdown_fence_language_collision`).

\newpage

9. Trust and security model

DreamGraph v1 ships the **trusted in-process plugin model**. This section is required reading before you publish or install a plugin.

9.1 The execution boundary

Plugins run **inside the daemon's Node process**. They share its heap, its file descriptors, and its operating-system permissions. The host:

- Enforces the manifest schema and capability gates at the API boundary;
- Mediates every host-owned effect (tool registration, resource registration, UI/policy/archetype/fence writes, event emission);
- Refuses to load anything not under `<instance>/plugins/` or `instance.json:plugins[]`;
- Refuses to load anything unless `DG_ALLOW_INPROCESS_PLUGINS=true` AND the plugin is explicitly `trusted: true`.

The host **cannot** prevent malicious plugin code from reaching `process`, `fs`, `child_process`, or any other Node global. Reaching around `ctx` is a trust violation and grounds for quarantine, not a sandbox escape.

Mental model: installing a DreamGraph plugin is closer to editing your `~/.bashrc` than to installing a browser extension. Only install plugins from sources you trust as much as you trust the daemon itself.

9.2 The two safety switches

Switch	Where	Default	Purpose
<code>DG_ALLOW_INPROCESS_PLUGINS</code>	engine.env / process env	<code>false</code>	Operator-wide kill switch. While off, no plugin loads.
<code>instance.json:plugins[].trusted</code>	per-plugin entry in <code>instance.json</code>	<code>false</code>	Per-plugin opt-in. Set with <code>dg plugin trust <id></code> .

Loading occurs **only** when both are true. The host emits `in_process_execution_disabled` and refuses the plugin otherwise.

9.3 The trust banner

Every successful in-process load prints exactly one line to the daemon log:

```
plugin <id>@<ver> loaded with FULL HOST TRUST (in-process)
```

If you do not see this line for your plugin, it did not load. Check the log for the matching `plugin.errorred` or capability-rejection event.

9.4 The event-loop watchdog

A `setImmediate` heartbeat samples Node event-loop lag. If the lag exceeds `DG_EVENT_LOOP_LAG_MS` (default 250 ms) while a plugin handler is on the call stack, the host:

1. Marks the handler `event_loop_stall`;
2. Waits for the event loop to become responsive;
3. Quarantines the plugin and emits `plugin.errorred{ reason: "event_loop_stall" }` with the measured lag.

This is detection, not preemption — the watchdog cannot interrupt synchronous CPU-bound code. It exists to make stalls visible *after* they happen so operators can disable the offending plugin.

9.5 What "quarantine" means

A quarantined plugin:

- Has all its registrations pruned (tools, resources, UI, policies, archetypes, fences).

- Stops receiving subscribed events.
- Remains visible in `system://plugins` with `status: "quarantined"` and the reason code.
- Cannot be reloaded until the operator runs `dg plugin enable <id>` (which re-trusts and re-attempts activation).

9.6 Coming in M8

Worker-thread isolation is planned for M8. Plugins declaring `runtime: "worker"` will be loaded into a `worker_threads.Worker`; the `PluginContext` is marshalled across `MessagePort`. Worker threads improve fault and event-loop isolation but are not, by themselves, a complete sandbox for untrusted marketplace code. The trusted in-process model is the only published contract until M8 ships.

\newpage

10. Installation and lifecycle

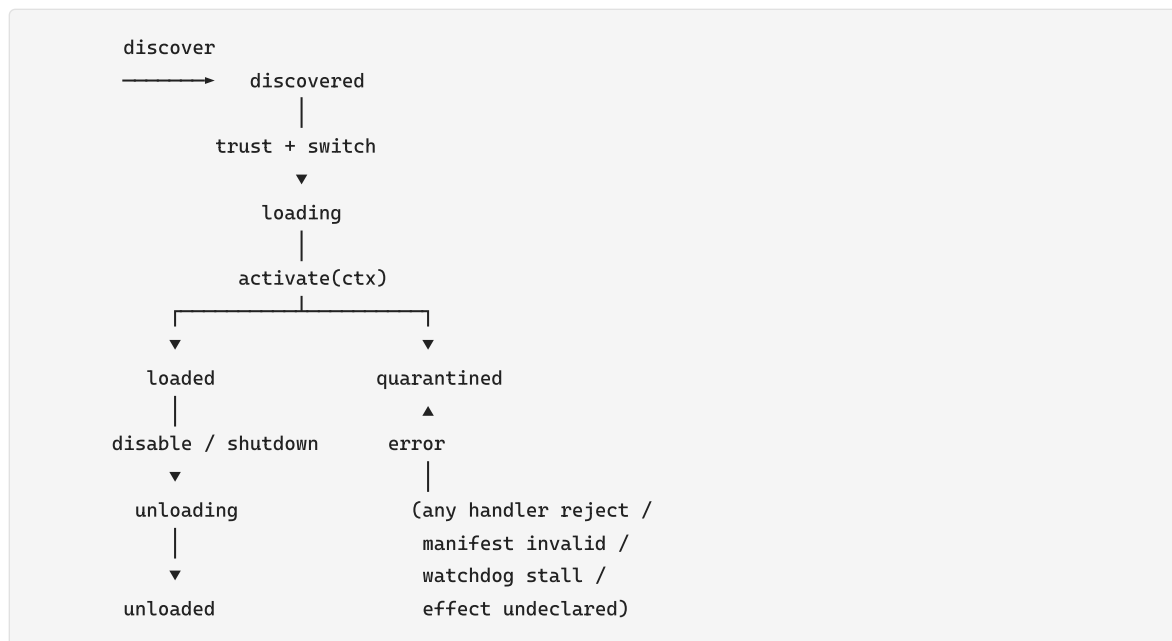
10.1 Discovery roots

The host scans exactly two roots:

1. `<instance>/plugins/*/plugin.json` — folder-style.
2. `<instance>/instance.json:plugins[]` — explicit `[{ path, trusted }]` entries.

Anything elsewhere on disk is ignored. There is no `node_modules` auto-scan.

10.2 The five lifecycle states



`system://plugins` exposes the current state for every plugin, plus the last seen effects, last error reason, and last activation timestamp.

10.3 The CLI

<code>dg plugin list <instance></code>	# all discovered plugins + state
<code>dg plugin inspect <instance> <plugin-id></code>	# full manifest + last telemetry
<code>dg plugin trust <instance> <plugin-id></code>	# set <code>instance.json:plugins[].trusted=true</code>

```
dg plugin enable <instance> <plugin-id>      # untrust → trust + re-activate
dg plugin disable <instance> <plugin-id>      # prune contributions, keep entry
```

After any change to the plugin's code (or to the SDK or host packages), restart the daemon: `dg restart <instance>`.

10.4 Hot reload

There is no automatic file-watcher. Each plugin is loaded once per daemon process. To pick up code changes:

1. Edit the plugin source.
2. (If you bundled it) re-build.
3. `dg plugin disable <instance> <plugin-id>` then `dg plugin enable <instance> <plugin-id>`. The host calls your `deactivate()` then re-imports the entry module and calls `activate(ctx)` again.

ESM module caching means the host bypasses the cache by appending a unique query-string to the import URL on re-enable. You do not have to do anything.

10.5 Shipping inside a workspace

If your plugin lives in the same npm workspace as the daemon (rare), declare it in the host's install pipeline so `scripts/install.ps1` packs it into `bin/vendor/`. Otherwise, ship the plugin folder with `plugin.json` and the compiled entry module — that is the entire deliverable.

10.6 Versioning and compatibility

- Bump `version` on every change that affects the manifest, the seam shape, or the public behavior of your tools/resources.
- Keep `engine.dreamgraph` narrow but accurate. The host emits a clear `engine_version_mismatch` rather than a runtime crash if the range excludes the current engine.
- The SDK follows semver. Stable event payload shapes never change in a backwards-incompatible way within a major. Experimental kinds may.

\newpage

11. Telemetry, debugging, and testing

11.1 The plugin telemetry kinds

Every plugin emits a uniform telemetry stream. Operators (and your tests) can subscribe to these on the bus to understand exactly what the host is doing on your behalf.

Kind	Emitted when
<code>plugin.loaded</code>	Manifest accepted, all seams wired.
<code>plugin.unloaded</code>	Graceful disable / shutdown.
<code>plugin.errorred</code>	Manifest rejection, capability violation, throw, watchdog stall.
<code>plugin.handler.started</code>	A handler/tool/action is about to run (correlation id).
<code>plugin.handler.completed</code>	Handler returned successfully (<code>duration_ms</code>).
<code>plugin.output.accepted</code>	Host wrote/forwarded/dispatched the plugin's output.
<code>plugin.output.rejected</code>	Host refused (carries <code>reason</code> , see reject reasons).

Every telemetry event carries `plugin_id`, `plugin_version`, and the seam involved. `plugin.output.rejected` additionally carries the `reason` code from the central enum and a human-readable `message`.

11.2 Reading telemetry while developing

Three options:

- **Daemon log** — every `plugin.errorred` and `plugin.output.rejected` is printed.
- **SSE** — `curl -N http://127.0.0.1:8010/explorer/api/events` streams the whole bus.
- **event_log.json** — under the instance data dir, the persistent record.

For tests, [tests/plugins/hello-events-e2e.test.ts](#) shows how to assert the exact telemetry sequence in a temp instance.

11.3 Testing

The SDK ships a phased test harness:

Phase	Surface	Use for
1	Manifest validation fixtures (schema-only)	unit tests; no runtime, no bus.
2	Seam mocks: fake <code>PluginContext</code> , stub bus, in-memory <code>system://plugins</code>	unit-test a tool handler or subscriber.
3	Full in-memory daemon harness (post-MVP, opens with M7)	E2E with real persistence and bus.

For Phase-1/2 in your own repo:

```
import { parsePluginManifest } from "@dreamgraph/sdk";
import manifest from "../plugin.json" assert { type: "json" };

it("manifest parses", () => {
  expect(() => parsePluginManifest(manifest)).not.toThrow();
});
```

For an end-to-end check, copy your plugin folder into a temp instance and assert against `ui_registry.json` / `plugin_policy_proposals.json` / `event_log.json`. The hello-events e2e test is a working template.

11.4 Common debugging recipes

- **Tool not appearing in `tools/list`?** The plugin probably did not load. Check the daemon log for the trust banner. If it's missing, check `DG_ALLOW_INPROCESS_PLUGINS` and `dg plugin list` (the entry must show `trusted: true`).
- **Tool args arrive as `undefined`?** Earlier engine versions had a bug where the MCP tool was registered with an empty Zod shape (the JSON Schema wasn't converted). Fixed in v8.3.0; if you see this, upgrade.
- **Handler throws `EventEmitter warning`?** You probably called `ctx.events.subscribe` inside a handler instead of inside `activate(ctx)`. All subscriptions should happen in `activate`.
- **`plugin.output.rejected{ reason: "effect_undeclared" }`?** You emitted a side-effect (event, registration) whose effect is not in the manifest's `expectedEffects`. Add the effect or remove the call.

\newpage

12. Best practices and pitfalls

12.1 Do

- **Name everything for the model, not the human.** ADR-128 — tool names must carry the dominant action and target. A model should not need your description to pick the right tool.
- **Declare every effect.** `expectedEffects` is read by operators and by the discipline engine; missing entries are a bug, not a convenience.
- **Use `forbiddenEffects` actively.** `write_internal_graph` and `modify_tensions` should be in every plugin's `forbiddenEffects` unless the plugin's entire reason for existing is graph mutation (rare).
- **Keep handlers fast.** Default timeout is 5 s. Anything slower is either awaiting I/O (use `ctx.signal`) or doing CPU work that belongs in a future schedule action.
- **Log through `ctx.logger`.** Operators see those lines. `console.log` lines do not propagate to the bus.
- **Treat the bus as at-most-once.** Reconstruct critical state from a resource read, not from your subscriber buffer.
- **Pin `engine.dreamgraph` accurately.** `>=9.0.0` is the right floor for v1.
- **Ship a README in your plugin folder.** `dg plugin inspect` shows it.

12.2 Don't

- **Don't import from `dreamgraph/src/...`.** The only stable import is `@dreamgraph/sdk`. Daemon internals change without notice.
- **Don't reach for `fs`, `process`, or `child_process`.** Even though the M3 model can't physically stop you, doing so is a trust violation and the operator may quarantine your plugin.
- **Don't rely on a shared global between handlers.** The host may call `deactivate` and re-`activate` between requests; rebuild state inside `activate(ctx)` so a re-enable is clean.
- **Don't poll on a `setInterval`.** Use the schedule action seam once M4 ships. Until then, recurring work belongs in the host.
- **Don't depend on plugin load order.** Plugins are independent. If your plugin needs another plugin's data, read its MCP resource explicitly.
- **Don't ship secrets in `plugin.json` or in the entry module.** The manifest is world-readable. Use the per-plugin config block (`config.schema`) and document where the operator should put credentials (typically `<instance>/config/<plugin-id>.json`).

12.3 Pitfalls (compendium)

Symptom	Likely cause
Plugin shows in <code>dg plugin list</code> but no banner	<code>DG_ALLOW_INPROCESS_PLUGINS</code> is false.
Banner appears, no contributions visible	Capability missing in <code>capabilities</code> array.
Tool args arrive empty	Bug in engine < 8.3.0; upgrade. Or tool registered with no <code>inputSchema</code> .
<code>plugin.output.rejected{ effect_undeclared }</code>	You emit an event/register a tool whose effect is not in <code>expectedEffects</code> .
<code>plugin.output.rejected{ tool_name_unprefixed }</code>	Tool name doesn't start with <code><plugin-id>.</code>
<code>plugin.output.rejected{ resource_uri_out_of_namespace }</code>	Resource URI doesn't start with <code>plugin://<plugin-id>/</code> .
<code>plugin.errorred{ event_loop_stall }</code>	Handler did synchronous CPU work > <code>DG_EVENT_LOOP_LAG_MS</code> .
<code>plugin.errorred{ handler_timeout }</code>	Handler didn't return inside the timeout; honor <code>ctx.signal</code> .
<code>plugin.errorred{ engine_version_mismatch }</code>	<code>engine.dreamgraph</code> semver excludes this engine.
Webhook never fires for your custom event	The webhook subscription's <code>events[]</code> doesn't list your kind.

12.4 The shipping checklist

Before publishing a plugin:

- ☐ Manifest validates against `parsePluginManifest`.
- ☐ Every tool/resource/UI/policy id is `<plugin-id>.*`.
- ☐ Every effect the plugin performs is in `expectedEffects`.
- ☐ `forbiddenEffects` contains everything the plugin guarantees not to do.
- ☐ README explains: install steps, required config, what each tool returns.
- ☐ Manifest tested with at least one manifest fixture test.
- ☐ Hello-world tool round-trips an argument over MCP.
- ☐ On `dg plugin disable`, all contributions are gone (manual `Get-Content` on the four data files confirms this).
- ☐ Trust banner appears on load; `system://plugins` shows `status: "loaded"`.

\newpage

13. Architect tabs

[<- Best practices](#) · [Reference: Architect tabs](#)

Trusted plugins can contribute first-class tabs to the daemon-served standalone Architect surface through `ctx.architect`. The browser boundary is declarative: plugin handlers run in the host process, while browser clients receive validated descriptors and JSON snapshots rendered by host-supported renderer kinds.

The initial renderer is `checklist`. Arbitrary browser JavaScript, HTML, remote scripts, inline scripts, iframes, and direct DOM callbacks are not supported.

13.1 Manifest declaration

Declare every tab statically in `plugin.json`:

```
{
  "capabilities": [
    "architect:register_tab",
    "architect:read_plan",
    "architect:write_plan_state",
    "architect:register_sidebar_summary"
  ],
  "expectedEffects": [
    "render_architect_tab",
    "read_architect_plan_projection",
    "write_architect_plan_state",
    "render_architect_sidebar_summary"
  ],
  "architectTabs": [
    {
      "id": "examples.action-checklist.checklist",
      "renderer": "checklist",
      "planConnectivity": "required"
    }
  ]
}
```

```
]
}
```

Tab ids MUST start with `<plugin-id>`. Runtime registration MUST match the manifest declaration.

13.2 Registering a tab

Use `defineArchitectTab()` from `@dreamgraph/sdk`, then register the result with `ctx.architect.tabs.register()`. The returned callback unregisters the contribution early; normal disable, unload, and reload paths remove active contributions automatically.

Plan state is daemon-owned and namespaced by instance, plugin id, plan id, tab type id, and key. Read and write it through `ctx.architect.planState`. Writes use revision tokens so stale browser actions fail instead of overwriting newer state.

```
const stored = await ctx.architect.planState.read("checklist", context);
const next = await ctx.architect.planState.write("checklist", value, {
  ...context,
  revision: stored?.revision ?? null,
});
```

Plan-bound operations require an explicit `planId`. A project-scope view does not silently bind to a selected or first plan.

13.3 Browser routes

Standalone Architect clients use versioned daemon routes under `/api/architect/v1/plugin-tabs` to list descriptors, fetch plan snapshots, and dispatch structured actions. Sidebar summaries and badges are projections from the same daemon snapshot. Store paths and plugin modules are never exposed to the browser.

13.4 Walkthrough

See [examples/action-checklist](#) for a complete reference plugin. It seeds Slice A-E checklist state on first load, toggles items through schema-validated actions, projects checklist progress and badges, persists state across reload, and unloads cleanly.

`ctx.ui.register()` remains a separate semantic metadata seam. It does not execute plugin browser code and does not register standalone Architect tabs.

[← Best practices](#) · [Reference: Architect tabs](#)

\newpage

Part II --- Plugin Reference Manual

\newpage

DreamGraph Plugin Reference Manual

Audience: Plugin authors and host implementors needing strict, normative tables. **Engine baseline:** v12.1.0 "Living DreamGraph". **Companion:** see the [Plugin Developer Guide](#) for task-oriented walkthroughs.

This reference is the source of truth. Where the guide and the reference disagree, the reference wins. Where the reference and the SDK source disagree, the SDK source wins (and a doc bug should be filed).

Sections

1. [Manifest schema](#) — every field, every constraint.
2. [Capabilities](#) — the locked vocabulary.
3. [Effects](#) — observable cognitive effects.
4. [Reject reasons](#) — every `plugin.output.rejected.reason`.
5. [PluginContext API](#) — TypeScript surface.
6. [Events](#) — stable kinds, experimental kinds, payload schemas.
7. [CLI](#) — `dg plugin` subcommands.
8. [Host configuration](#) — env vars and `instance.json`.
9. [Telemetry events](#) — the `plugin.*` stream.
10. [Architect tabs](#) — declarative tab contract, plan state, routes, lifecycle, and telemetry.

Conventions

- **MUST / MUST NOT / SHOULD** are used in the RFC 2119 sense.
- Tables are exhaustive unless explicitly marked "non-exhaustive".
- Source-code references link to the canonical file at the repo root.

\newpage

Manifest schema (normative)

Top-level shape

```
interface PluginManifest {
  id: string; // PluginIdSchema
  version: string; // ≥ 1 char
  displayName: string; // ≥ 1 char
  description?: string;
  author?: string;
  license?: string;
  engine: { dreamgraph: string }; // semver range
  main: string; // path to ESM entry
  intent: string;
  expectedEffects: PluginEffect[]; // ≥ 1
  forbiddenEffects: PluginEffect[]; // default []
  capabilities: PluginCapability[]; // ≥ 1
  tools: { name: string }[]; // default []
  resources: { uriNamespace: string }[]; // default []
  ui: { id: string }[]; // default []
  policies: { id: string }[]; // default []
  archetypeProviders: { id: string }[]; // default []
  markdownFences: { language: string }[]; // default []
  architectTabs: PluginManifestArchitectTab[]; // default []
  policy: PluginPolicy; // default { phasePermissions: [],
    writableFiles: [] }
  config?: { schema?: string };
}
```

The schema is **strict**: unknown top-level fields are rejected with `manifest_invalid`.

Field-level constraints

Field	Constraint	Reject reason on violation
id	<code>^[a-z0-9][a-z0-9.-]*[a-z0-9]\$, ≥ 3 chars</code>	manifest_invalid
version	non-empty string (semver recommended)	manifest_invalid
engine.dreamgraph	semver range; host engine version MUST satisfy it	engine_version_mismatch
main	non-empty path; resolved relative to plugin folder	manifest_invalid
intent	≥ 1 char	manifest_invalid
expectedEffects	≥ 1 entry, each in <code>PluginEffect</code>	manifest_invalid
forbiddenEffects	each in <code>PluginEffect</code>	manifest_invalid
expectedEffects n forbiddenEffects	MUST be empty	manifest_invalid
capabilities	≥ 1 entry, each in <code>PluginCapability</code>	manifest_invalid
tools[].name	<code><plugin-id>.<tool> — ^[a-z0-9][a-z0-9.-]*\.[a-z0-9][a-z0-9_-]*\$</code>	tool_name_unprefixed
resources[].uriNamespace	<code>^plugin:\/\<plugin-id>(?:\/.*)*\$</code>	resource_uri_out_of_namespace
ui[].id	lowercase <code>[a-z0-9._-]+</code>	ui_id_unprefixed (at register time)
policies[].id	lowercase <code>[a-z0-9._-]+</code>	manifest_invalid
archetypeProviders[].id	lowercase <code>[a-z0-9._-]+</code>	manifest_invalid
markdownFences[].language	lowercase <code>[a-z0-9._-]+</code>	markdown_fence_language_invalid
architectTabs[].id	lowercase <code>[a-z0-9._-]+</code> , MUST start with <code><plugin-id>.</code>	manifest_invalid
architectTabs[].renderer	currently <code>checklist</code> only	manifest_invalid
architectTabs[].planConnectivity	<code>`required</code>	optional
policy.phasePermissions	each <code>∈ `analysis</code>	execution
policy.writableFiles	array of strings	manifest_invalid

Validation entry point

```
import { parsePluginManifest, PluginManifestSchema } from "@dreamgraph/sdk";

const manifest = parsePluginManifest(JSON.parse(plugin_json_text));
// throws ZodError with reason "manifest_invalid" on failure
```

Example

See [examples/hello-events/plugin.json](#) for a manifest exercising every M6 seam and [examples/action-checklist/plugin.json](#) for a standalone Architect tab manifest.

Capabilities (normative)

A capability is a coarse permission a plugin MUST declare to use a seam. The host validates the manifest against this enum at load time, then re-checks at each seam call site.

Capability	Allows (positive)	Denies (with reason)
<code>events:read</code>	<code>ctx.events.subscribe(stable_kind, fn)</code>	subscribing to experimental kind → <code>event_kind_experimental</code> ; kind not in declared subscription list → <code>event_kind_not_declared</code>
<code>events:read:experimental</code>	<code>defineExperimentalEventHandler(...)</code>	unless <code>instance.json:experimental_plugin</code> true → <code>experimental_events_disabled</code>
<code>events:emit</code>	<code>ctx.events.emit(kind, payload)</code>	<code>emit_event</code> not in <code>expectedEffects</code> → <code>effect_undeclared</code> ; missing capability → <code>emit_capability_missing</code>
<code>tools:register</code>	<code>ctx.tools.register({ name: "<plugin-id>.<x>", ... })</code>	name without <code><plugin-id></code> . prefix → <code>tool_name_unprefixed</code> ; collision with or other plugin → <code>tool_name_collision</code>
<code>resources:register</code>	<code>ctx.resources.register({ uriNamespace: "plugin://<id>/...", ... })</code>	URI not in namespace → <code>resource_uri_out_of_namespace</code>
<code>resources:read</code>	(reserved) read another plugin's resource	always denied today
<code>ui:register</code>	<code>ctx.ui.register({ id: "<plugin-id>.<x>", ... })</code>	id not prefixed → <code>ui_id_unprefixed</code>
<code>policy:propose</code>	<code>ctx.policies.propose({ id, ... })</code> (tagged source: <code>plugin:<id></code>)	proposal admits <code>weakens: true</code> or weakens existing rule → <code>policy_weakening_rejected</code>
<code>archetypes:provide</code>	<code>ctx.archetypes.registerProvider({ id, inline ... })</code>	url
<code>markdown:register_fence</code>	<code>ctx.markdownFences.register({ language, ... })</code>	language regex fails → <code>markdown_fence_language_invalid</code> ; collision → <code>markdown_fence_language_collision</code>
<code>schedule:register_action</code>	(reserved, M4) <code>defineScheduleAction(...)</code>	always denied today
<code>webhooks:emit</code>	(reserved, post-MVP) <code>ctx.webhooks.dispatch(...)</code>	always denied today (use core webhook subscriptions instead)
<code>providers:register</code>	(reserved, post-1.0) provider adapter seam	always denied today
<code>architect:register_tab</code>	<code>ctx.architect.tabs.register(definition)</code>	missing capability → <code>architect_tab_capability_missing</code> ; undeclared tab → <code>architect_tab_undeclared</code>
<code>architect:read_plan</code>	<code>ctx.architect.planState.read(key, context)</code>	missing explicit plan → <code>architect_plan_required</code> ; missing plan → <code>architect_plan_not_found</code>
<code>architect:write_plan_state</code>	<code>ctx.architect.planState.write(key, value, context)</code>	missing capability → <code>architect_plan_state_capability_missing</code> ; stale revision → <code>architect_revision_stale</code>

Capability	Allows (positive)	Denies (with reason)
<code>architect:register_sidebar_summary</code>	project sidebar summary and badges from a tab snapshot	undeclared effect -> <code>effect_undeclared</code>

A capability not present in `manifest.capabilities` MUST cause the matching seam call to be refused with `<seam>_capability_missing`. The host MUST NOT silently drop the call.

\newpage

Effects (normative)

An **effect** describes an observable cognitive impact a plugin claims it will have (`expectedEffects`) or guarantees it will not have (`forbiddenEffects`). The host enforces both at every host-mediated effect call:

- An effect not in `expectedEffects` → `plugin.output.rejected{ effect_undeclared }`.
- An effect in `forbiddenEffects` → `plugin.output.rejected{ effect_forbidden }`, even if the matching capability would otherwise allow it.

Effect	Meaning	Required capability
<code>read_events</code>	Subscribe to the bus (any kind).	<code>events:read</code> (or <code>:experimental</code>)
<code>read_internal_graph</code>	Read internal-tier files (graph, validated edges, tensions).	(host-mediated, no plugin write API)
<code>read_external_state</code>	Read external-tier files (features, archetypes, ADRs, UI registry).	(host-mediated)
<code>write_internal_graph</code>	Mutate the graph (promote candidates, edit edges).	always forbidden
<code>modify_tensions</code>	Create or resolve tensions.	always forbidden
<code>emit_resource</code>	Register or update an MCP resource.	<code>resources:register</code>
<code>emit_tool</code>	Register an MCP tool.	<code>tools:register</code>
<code>emit_event</code>	Publish events onto the bus.	<code>events:emit</code>
<code>emit_webhook</code>	(reserved) outbound HTTP via webhook worker.	<code>webhooks:emit</code>
<code>register_schedule_action</code>	(reserved) custom schedule action.	<code>schedule:register_action</code>
<code>invoke_llm</code>	Call the host LLM proxy.	(post-M4)
<code>mutate_ui_registry</code>	Register/update UI elements.	<code>ui:register</code>
<code>propose_policy</code>	Append discipline rules.	<code>policy:propose</code>
<code>provide_archetypes</code>	Feed federation archetypes.	<code>archetypes:provide</code>
<code>render_markdown_fence</code>	Declare a webview markdown fence.	<code>markdown:register_fence</code>
<code>register_provider_adapter</code>	(reserved, post-1.0) provider adapter.	<code>providers:register</code>

Effect	Meaning	Required capability
<code>render_architect_tab</code>	Project a host-rendered standalone Architect tab.	<code>architect:register_tab</code>
<code>read_architect_plan_projection</code>	Load explicit plan-bound Architect state.	<code>architect:read_plan</code>
<code>write_architect_plan_state</code>	Persist daemon-owned namespaced Architect state.	<code>architect:write_plan_state</code>
<code>render_architect_sidebar_summary</code>	Project sidebar summary and badge data.	<code>architect:register_sidebar_summary</code>

Always-forbidden effects

`write_internal_graph` and `modify_tensions` are not exposed by `ctx` at all. A plugin that lists them in `expectedEffects` will fail manifest validation because the matching capability does not exist. They are listed here for completeness so plugin authors know to put them in `forbiddenEffects` as defensive intent.

Effect-to-seam matrix

Seam call	Effect required in <code>expectedEffects</code>
<code>ctx.tools.register</code>	<code>emit_tool</code>
<code>ctx.resources.register</code>	<code>emit_resource</code>
<code>ctx.events.subscribe</code>	<code>read_events</code>
<code>ctx.events.emit</code>	<code>emit_event</code>
<code>ctx.ui.register</code>	<code>mutate_ui_registry</code>
<code>ctx.policies.propose</code>	<code>propose_policy</code>
<code>ctx.archetypes.registerProvider</code>	<code>provide_archetypes</code>
<code>ctx.markdownFences.register</code>	<code>render_markdown_fence</code>
<code>ctx.architect.tabs.register</code>	<code>render_architect_tab</code>
<code>ctx.architect.planState.read</code>	<code>read_architect_plan_projection</code>
<code>ctx.architect.planState.write</code>	<code>write_architect_plan_state</code>

\newpage

Reject reasons (normative)

Every `plugin.output.rejected` event carries one of the following `reason` codes. The set is closed and exported as `PluginRejectReason` from the SDK.

Per seam

`tools`

Reason	Meaning
<code>tool_name_unprefixed</code>	Tool name does not start with <code><plugin-id>.</code>
<code>tool_name_collision</code>	Tool name collides with a built-in or another plugin.
<code>tool_capability_missing</code>	<code>tools:register</code> not in <code>manifest.capabilities</code> .

resources

Reason	Meaning
<code>resource_uri_out_of_namespace</code>	URI does not start with <code>plugin://<plugin-id>/</code> .
<code>resource_capability_missing</code>	<code>resources:register</code> not declared.

events

Reason	Meaning
<code>event_kind_not_declared</code>	Subscriber kind not listed in the manifest's declared subscription set.
<code>event_kind_experimental</code>	Subscriber kind is experimental and the plugin lacks <code>events:read:experimental</code> .
<code>experimental_events_disabled</code>	Instance has not opted into <code>experimental_plugins: true</code> .
<code>event_capability_missing</code>	<code>events:read</code> (or <code>:experimental</code>) not declared.
<code>effect_undeclared</code>	Plugin emitted an event whose effect is not in <code>expectedEffects</code> .
<code>effect_forbidden</code>	Plugin emitted an event whose effect is in <code>forbiddenEffects</code> .
<code>emit_capability_missing</code>	<code>events:emit</code> not declared.

ui

Reason	Meaning
<code>ui_id_unprefixed</code>	Element id does not start with <code><plugin-id>.</code>
<code>ui_capability_missing</code>	<code>ui:register</code> not declared.

policy

Reason	Meaning
<code>policy_weakening_rejected</code>	Proposal weakens an existing rule (<code>weakens: true</code> or detected).
<code>policy_capability_missing</code>	<code>policy:propose</code> not declared.

archetypes

Reason	Meaning
<code>archetype_schema_invalid</code>	Archetype payload fails schema validation.
<code>archetype_capability_missing</code>	<code>archetypes:provide</code> not declared.

markdown_fences

Reason	Meaning
<code>markdown_fence_capability_missing</code>	<code>markdown:register_fence</code> not declared.
<code>markdown_fence_language_collision</code>	Another plugin already owns the fence language.
<code>markdown_fence_language_invalid</code>	Language regex fails.

schedule (reserved)

`action_id_collision`, `action_schema_missing`, `schedule_capability_missing`.

webhooks (reserved)

`webhook_unsigned`, `webhook_target_blocked`, `webhook_capability_missing`.

llm (post-M4)

`llm_budget_exceeded`, `llm_disabled`, `llm_capability_missing`.

providers (reserved)

`provider_capability_missing`.

architect

Reason	Meaning
<code>architect_tab_capability_missing</code>	<code>architect:register_tab</code> not declared.
<code>architect_plan_state_capability_missing</code>	Required Architect plan-state capability not declared.
<code>architect_tab_id_unprefixed</code>	Tab id does not start with <code><plugin-id>.</code>
<code>architect_tab_collision</code>	Another active contribution owns the tab id.
<code>architect_tab_undeclared</code>	Runtime tab registration does not match <code>manifest.architectTabs</code> .
<code>architect_action_unknown</code>	Requested action id is not registered for the tab.
<code>architect_action_schema_invalid</code>	Action payload fails the registered JSON schema.
<code>architect_plan_required</code>	A plan-bound operation omitted its explicit plan id.
<code>architect_plan_not_found</code>	The requested plan does not exist.
<code>architect_revision_stale</code>	State write revision does not match current daemon state.
<code>architect_tab_unavailable</code>	Tab descriptor or handler is no longer active.

Host execution boundary

Reason	Meaning
<code>in_process_execution_disabled</code>	<code>DG_ALLOW_INPROCESS_PLUGINS</code> is false or plugin not <code>trusted</code> .
<code>plugin_quarantined</code>	Plugin previously errored and is held until manual re-enable.
<code>event_loop_stall</code>	Watchdog detected a stall (lag > <code>DG_EVENT_LOOP_LAG_MS</code>).
<code>handler_timeout</code>	Handler did not return within the per-handler timeout.
<code>handler_threw</code>	Handler threw or returned a rejected promise.

Reason	Meaning
manifest_invalid	Manifest failed <code>PluginManifestSchema</code> validation.
engine_version_mismatch	Current engine version is outside <code>manifest.engine.dreamgraph</code> range.

Helpers

```
import { rejectSeamOf, isPluginRejectReason } from "@dreamgraph/sdk";

rejectSeamOf("tool_name_unprefixed");    // "tools"
isPluginRejectReason("nope");            // false
```

\newpage

PluginContext API (normative)

Top-level

```
interface PluginContext {
  readonly plugin:      { id: string; version: string };
  readonly instance:    { uuid: string };
  readonly logger:      PluginLogger;
  readonly events:      PluginEventEmitSurface;
  readonly tools:       PluginToolsSurface;
  readonly resources:   PluginResourcesSurface;
  readonly ui:          PluginUiSurface;
  readonly policies:    PluginPoliciesSurface;
  readonly archetypes:  PluginArchetypesSurface;
  readonly markdownFences: PluginMarkdownFencesSurface;
  readonly architect:   PluginArchitectSurface;
  readonly signal:      AbortSignal;
}
```

Surfaces

```
interface PluginLogger {
  debug(message: string, meta?: object): void;
  info(message: string, meta?: object): void;
  warn(message: string, meta?: object): void;
  error(message: string, meta?: object): void;
}

interface PluginEventEnvelope {
  readonly kind: string;
  readonly payload: unknown;
}

type PluginEventHandler = (e: PluginEventEnvelope) => void | Promise<void>;

interface PluginEventEmitSurface {
  emit(kind: string, payload: unknown): void;
}
```

```

    subscribe(kind: string, handler: PluginEventHandler): () => void;
}

interface PluginToolsSurface { register(d: ToolDefinition): () => void; }
interface PluginResourcesSurface { register(d: ResourceDefinition): () => void; }
interface PluginUiSurface { register(d: UiElementDefinition): () => void; }
interface PluginPoliciesSurface { propose(p: PolicyProposal): () => void; }
interface PluginArchetypesSurface { registerProvider(d: ArchetypeProviderDefinition): () => void; }

interface PluginMarkdownFencesSurface { register(d: MarkdownFenceDefinition): () => void; }
interface PluginArchitectSurface {
  tabs: { register(d: ArchitectTabTypeDefinition): () => void };
  planState: {
    read<T>(key: string, context: ArchitectPlanContext): Promise<ArchitectStoredState<T> | null>;
    write<T>(key: string, value: T, context: ArchitectPlanWriteContext): Promise<ArchitectStoredState<T>>;
  };
}

```

The complete Architect tab definition, lifecycle, route, capability, and effect contract is in [Architect tabs](#).

Seam definition shapes

ToolDefinition

```

interface ToolDefinition<Input = unknown, Output = unknown> {
  name: string; // <plugin-id>.<tool>
  description: string;
  inputSchema: JsonSchema; // converted to Zod by host
  expectedEffects: PluginEffect[]; // must include "emit_tool"
  forbiddenEffects?: PluginEffect[];
  handler: (input: Input, ctx: { pluginId: string; signal?: AbortSignal })
    => Output | Promise<Output>;
}

```

ResourceDefinition

```

interface ResourceDefinition<Params = Record<string, unknown>, Output = unknown> {
  uriNamespace: string; // plugin://<plugin-id>/...
  name: string;
  description?: string;
  outputSchema?: JsonSchema;
  expectedEffects: PluginEffect[]; // must include "emit_resource"
  forbiddenEffects?: PluginEffect[];
  handler: (req: { uri: string; params?: Params },
    ctx: { pluginId: string; signal?: AbortSignal })
    => Output | Promise<Output>;
}

```

UiElementDefinition

```

type UiElementCategory =
  | "data_display" | "data_input" | "navigation"
  | "feedback" | "layout" | "action" | "composite";

```

```

interface UiElementDefinition {
  id:      string;           // starts with <plugin-id>.
  name:    string;
  purpose: string;
  category: UiElementCategory;
  inputs:  UiElementInput[];
  outputs: UiElementOutput[];
  interactions: UiElementInteraction[];
  children?: string[];
  implementations?: UiElementImplementation[];
  used_by?:  string[];
  tags?:     string[];
  expectedEffects?: PluginEffect[]; // typically ["mutate_ui_registry"]
}

```

PolicyProposal

```

interface PolicyProposal {
  id:      string;
  title:   string;
  rationale: string;
  applies_to: string[];
  phases:  PhasePermission[];
  severity: "block" | "warn" | "info";
  weakens?: false; // MUST be false; reserved
}

```

ArchetypeProviderDefinition

```

interface ArchetypeRecord {
  id:      string;
  name:    string;
  summary: string;
  tags?:   string[];
  payload?: unknown;
}

interface ArchetypePayload {
  source:   string;
  version:  string;
  archetypes: ArchetypeRecord[];
}

interface ArchetypeProviderDefinition {
  id:      string;
  name:    string;
  url?:    string;           // reserved for remote polling
  poll_interval_s?: number;
  inline?: ArchetypePayload;
  fetch?:  () => Promise<ArchetypePayload>;
}

```

MarkdownFenceDefinition

```
interface MarkdownFenceDefinition {
  language:    string;           // /^[a-z0-9][a-z0-9._-]*$/
  label:       string;
  description: string;
  platforms?: Array<"webview" | "cli" | "web">;
}
```

Identity helpers

The SDK ships these zero-cost factories purely for IDE inference:

```
import {
  defineTool, defineResource, defineUiElement,
  definePolicyProposal, defineArchetypeProvider, defineMarkdownFence,
  definePlugin,
} from "@dreamgraph/sdk";
```

Each `define*` simply returns its argument with the matching type.

\newpage

Events (normative)

Envelope

```
interface EventEnvelope {
  seq:      number;
  kind:     string;
  affected_ids: string[];
  etag:     string | null;
  ts:       string;           // ISO-8601
  payload?: Record<string, unknown>;
}
```

Stable kinds

Imported from `@dreamgraph/sdk` :

- `snapshot.changed`
- `cache.invalidated`

Stable kinds MUST remain backwards-compatible within a major SDK version. Subscriptions require `events:read` .

Experimental kinds

Imported from `@dreamgraph/sdk/events/experimental` . Subscription requires `events:read:experimental` AND `instance.json:experimental_plugins: true` .

- `dream.cycle.completed`
- `tension.created`
- `tension.resolved`
- `candidate.added`

- candidate.promoted
- candidate.rejected
- audit.appended
- schedule.executed
- schedule.timed_out
- schedule.paused
- nightmare.cycle.completed
- archetype.imported
- archetype.exported
- narrative.chapter.appended

Selected payload schemas

schedule.executed

```
interface SchedulePayload {
  schedule_id: string;
  schedule_name: string;
  action: string;
  execution_id: string;
  success: boolean;
  duration_ms: number;
  status: string;
  error: string | null;
}
```

schedule.timed_out

```
interface ScheduleTimeoutPayload {
  schedule_id: string;
  schedule_name: string;
  action: string;
  error: string;
}
```

schedule.paused

```
interface SchedulePausedPayload {
  schedule_id: string;
  schedule_name: string;
  action: string;
  reason: "error_streak";
  error_count: number;
}
```

nightmare.cycle.completed

```
interface NightmarePayload {
  cycle_number: number;
  strategy: string;
  threats_found: number;
  attack_surfaces: number;
  summary: { critical: number; high: number; medium: number; low: number };
}
```

```
duration_ms:    number;
}
```

`archetype.exported`

```
interface ArchetypeExportedPayload {
  archetypes_exported: number;
  file_path:          string;
  instance_id:         string;
  timestamp:           string;
}
```

`archetype.imported`

```
interface ArchetypeImportedPayload {
  archetypes_imported: number;
  archetypes_skipped:  number;
  tensions_created:    number;
  source_instance:     string;
  timestamp:            string;
}
```

`narrative.chapter.appended`

```
interface NarrativeChapterPayload {
  chapter_number: number;
  title:          string;
  cycle_range:    [number, number];
  generated_at:   string;
}
```

Plugin-defined kinds

Plugins MAY emit any kind not in the host taxonomy. The SDK does not enforce naming, but the convention `<plugin-id>.<event_name>` matches the pattern used for tools, resources, and UI ids.

Replay

The SSE stream (`/explorer/api/events`) keeps a 500-event ring buffer. The persistent record is `<instance>/data/event_log.json` with rotation. The plugin contract is **at-most-once**: do not rely on receiving every event.

\newpage

dg plugin CLI (normative)

All commands take the instance name (or UUID) as the first positional argument.

```
dg plugin list <instance>
```

Lists every discovered plugin with its current state.

```
dg plugin list dreamgraph
```

Output columns: `id`, `version`, `state`, `runtime`, `trusted`, `last_loaded_at`.

dg plugin inspect <instance> <plugin-id>

Prints the full validated manifest, last seen effects, last activation timestamp, last error reason (if any), and the path to the plugin folder.

dg plugin trust <instance> <plugin-id>

Sets `trusted: true` on the plugin's entry in `instance.json:plugins[]`. Does not load or restart the daemon. Run `dg restart <instance>` to apply.

dg plugin enable <instance> <plugin-id>

Marks the plugin trusted (if not already) and triggers an in-place activate-cycle: calls `deactivate()` if the plugin is currently loaded, then re-imports the entry module (cache-busted) and calls `activate(ctx)`.

dg plugin disable <instance> <plugin-id>

Calls `deactivate()`, prunes every contribution registered through `ctx` (tools, resources, UI, policy proposals, archetype providers, markdown fences), and unsubscribes every event handler. The plugin entry remains in `instance.json` so a future `enable` reuses the same identity.

Notes

- The `dg plugin` family does not invoke MCP tools or read MCP resources. Use any MCP client (VS Code extension, Claude Desktop, `mcp inspector`) for that.
- `dg plugin trust` is idempotent. Repeated runs **must not** create duplicate entries in `instance.json:plugins[]` (a known issue in earlier engine versions; v8.3.0 deduplicates on write).
- All commands exit non-zero on error; standard `--json` flag is reserved for a future release.

\newpage

Host configuration (normative)

Environment variables

Variable	Default	Effect
DG_ALLOW_INPROCESS_PLUGINS	false	Master kill switch for in-process plugin execution. While <code>false</code> , no plugin loads even if <code>trusted: true</code> .
DG_EVENT_LOOP_LAG_MS	250	Watchdog threshold (milliseconds). Stalls above this while a plugin handler is on the call stack are reported as <code>event_loop_stall</code> .
DG_PLUGIN_HANDLER_TIMEOUT_MS	5000	Per-handler wall-clock budget. Exceeded handlers emit <code>handler_timeout</code> and quarantine the plugin.

Variable	Default	Effect
DG_PLUGIN_LLM_BUDGET_PCT	10	(post-M4) Percentage of host LLM budget reserved for plugins as a slice.
WEBHOOK_FAST_BACKOFF	unset	When set, replaces the spec backoff schedule with 1s/2s/4s/8s/16s for live testing.

The Plugin System block in all three engine.env templates (default , ollama , lmstudio) lists DG_ALLOW_INPROCESS_PLUGINS with the safety wording:

```
# --- Plugin System ---
# DG_ALLOW_INPROCESS_PLUGINS=true    # Enable in-process plugins (potentially unsafe, use with
trusted code only)
```

instance.json

```
{
  "uuid": "...",
  "experimental_plugins": false,
  "plugins": [
    { "path": "plugins/examples.hello-events", "trusted": true },
    { "path": "plugins/acme.changelog",          "trusted": false }
  ]
}
```

Field	Type	Effect
experimental_plugins	boolean (default false)	Required for any plugin holding events:read:experimental .
plugins[].path	string	Absolute or instance-relative path to the plugin folder.
plugins[].trusted	boolean (default false)	Per-plugin trust opt-in. Combined AND with DG_ALLOW_INPROCESS_PLUGINS .
plugins[].llm_budget	number (post-M4)	Override the per-plugin LLM sub-quota.

The host MUST de-duplicate plugins[] entries by path on write to prevent the duplicate-entry pathology that affected pre-8.3.0 engines after multiple dg plugin trust invocations.

Discovery roots (recap)

The host scans exactly two locations:

1. <instance>/plugins/*/plugin.json (folder-style auto-discovery).
2. instance.json:plugins[].path (explicit registration).

There is no node_modules scan. There is no auto-discovery from outside the instance directory.

\newpage

Telemetry events (normative)

The host emits a stable `plugin.*` stream. Every payload includes `plugin_id` , `plugin_version` , and ISO-8601 `occurred_at` .

Event kinds

Kind	Purpose
<code>plugin.loaded</code>	Successful activation, runtime, trust state, and capabilities.
<code>plugin.unloaded</code>	Shutdown, disable, reload, or quarantine cleanup.
<code>plugin.errorred</code>	Host boundary or handler failure.
<code>plugin.handler.started</code>	Timed handler invocation start.
<code>plugin.handler.completed</code>	Timed handler invocation completion with <code>ok</code> .
<code>plugin.output.accepted</code>	Host accepted a mediated effect.
<code>plugin.output.rejected</code>	Host refused a mediated effect or registration.

Handler seams

`plugin.handler.started` and `plugin.handler.completed` use:

```
type PluginHandlerSeam =
  | "tool"
  | "resource"
  | "event"
  | "schedule_action"
  | "architect_tab"
  | "architect_plan_state";
```

Both carry `correlation_id` , `seam` , and `target` . Completion also carries `duration_ms` and `ok` .

Accepted output seams

```
type PluginAcceptedOutputSeam =
  | "tool"
  | "resource"
  | "event"
  | "schedule_action"
  | "webhook"
  | "ui"
  | "archetype"
  | "policy"
  | "markdown_fence"
  | "architect_tab"
  | "architect_plan_state";
```

Accepted output carries `correlation_id` , `effect` , `seam` , and optional `target` .

Rejected output seams

Rejected output supports the accepted output seams plus `host` . It carries `reason: PluginRejectReason` , optional `correlation_id` , optional `target` , and optional human-readable `detail` .

Architect tab registration, plan-state access, snapshot loading, and action dispatch therefore use the normal host-owned telemetry stream. They do not create a browser-only audit channel.

Stability

The seven event kinds above are stable SDK contracts. Payloads may grow additively within a major SDK version.

\newpage

Architect tabs (normative)

<- Index · Sources: [packages/sdk/src/seams/architect.ts](#), [packages/host/src/context.ts](#)

Architect tabs are trusted host-side plugin contributions projected into the daemon-served standalone Architect browser surface. Browser clients MUST receive declarative descriptors and JSON snapshots only. They MUST NOT load plugin modules or receive raw state-store paths.

Manifest declaration

```
interface PluginManifestArchitectTab {
  id: string; // MUST start with <plugin-id>.
  renderer: "checklist";
  planConnectivity: "required" | "optional" | "none";
}
```

A manifest with any `architectTabs` entry MUST declare `architect:register_tab`, `architect:read_plan`, `render_architect_tab`, and `read_architect_plan_projection`. Runtime registration MUST match a manifest declaration.

Context API

```
interface PluginArchitectSurface {
  tabs: { register(definition: ArchitectTabTypeDefinition): () => void };
  planState: {
    read<T>(key: string, context: ArchitectPlanContext): Promise<ArchitectStoredState<T> | null>;
    write<T>(key: string, value: T, context: ArchitectPlanWriteContext):
      Promise<ArchitectStoredState<T>>;
  };
}
```

`planState` is daemon-owned and namespaced by instance, plugin id, plan id, tab type id, and key. Writes MUST include the expected revision or `null` for initial creation. State revisions are rejected.

Tab definition

```
interface ArchitectTabTypeDefinition<TState = unknown, TAction = unknown> {
  id: string;
  title: string;
  icon?: string;
  renderer: "checklist";
  planConnectivity: "required" | "optional" | "none";
  stateSchema: Record<string, unknown>;
}
```

```

actions: readonly { id: string; inputSchema: Record<string, unknown> }[];
sidebarSummary?: { kind: "checklist-progress" };
badges?: readonly { id: string; kind: "count" | "status" }[];
loadState(context: ArchitectPlanContext): Promise<TState> | TState;
handleAction?(action: TAction, context: ArchitectPlanActionContext): Promise<TState> | TState;
}

```

Capabilities and effects

Capability	Purpose
architect:register_tab	Register a manifest-declared Architect tab type.
architect:read_plan	Read explicit plan-bound state and projection context.
architect:write_plan_state	Persist daemon-owned namespaced plan state.
architect:register_sidebar_summary	Project a sidebar summary from the tab snapshot.

Effect	Purpose
render_architect_tab	Project a host-rendered Architect tab.
read_architect_plan_projection	Load explicit plan-bound snapshot state.
write_architect_plan_state	Persist namespaced plan state.
render_architect_sidebar_summary	Project sidebar summary and badge data.

Lifecycle and telemetry

Unload, disable, and reload remove active descriptors and handlers while retaining persisted plan state. Registration and invocation use the normal `plugin.output.accepted`, `plugin.output.rejected`, `plugin.handler.started`, and `plugin.handler.completed` telemetry contracts with `architect_tab` or `architect_plan_state` seams.

Browser contract

Versioned routes live under `/api/architect/v1/plugin-tabs`. Actions are structured daemon commands. The daemon validates plugin availability, explicit plan scope, action schema, and revision before returning a new snapshot.

`ctx.ui.register()` is metadata-only and is not an executable browser UI escape hatch.